

regarding-prediction-pipeline

March 6, 2025

```
[1]: import os
import numpy as np
import pandas as pd
from PIL import Image
from torch.utils.data import Dataset, DataLoader
import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as transforms
from sklearn.metrics import f1_score, accuracy_score
from torch.optim.lr_scheduler import StepLR
from sklearn.model_selection import train_test_split
from tqdm import tqdm
```

```
[11]: import pandas as pd
test = pd.read_csv('/kaggle/input/detect-ai-vs-human-generated-images/test.csv')
```

```
[ ]: import os
path = '/kaggle/input/ai-vs-human-generated-dataset/'
test_file_list = [os.path.join(path, fname) for fname in test['id']]
test_file_list_updated = []

for i in test_file_list:
    test_file_list_updated.append(i)
```

```
[ ]:
```

```
[13]: # test dataloader
# Dataset class for inference (validation and test)

# Validation and Test transforms

val_test_transforms = transforms.Compose([
```

```

transforms.Resize((224, 224)), # Resize the test image to 224x224
transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

class TestAllImageDataset(Dataset):
    def __init__(self, file_list, transform=None):
        self.file_list = file_list
        self.transform = transform

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, os.path.basename(img_path) # Return image and filename

test_dataset = TestAllImageDataset(file_list=test_file_list_updated,
    ↪transform=val_test_transforms)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False,
    ↪num_workers=0)

```

[14]: *# Load pretrained ConvNeXt Base model*

```

from torch.optim.lr_scheduler import OneCycleLR

model = models.convnext_large(weights="DEFAULT")

# Freeze all layers initially
for param in model.features.parameters():
    param.requires_grad = False

# Unfreeze the last two stages
for param in model.features[-2:].parameters():
    param.requires_grad = True

# Replace the classifier head with a custom one
model.classifier = nn.Sequential(
    nn.AdaptiveAvgPool2d((1, 1)),
    nn.Flatten(),
    nn.BatchNorm1d(1536), # Change from 1024 to 1536
    nn.Linear(1536, 512), # Change input size from 1024 to 1536
    nn.ReLU(),

```

```

    nn.Dropout(0.5),
    nn.Linear(512, 2)
)
# Move the model to gpu
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

```

Downloading: "https://download.pytorch.org/models/convnext_large-ea097f82.pth"
to /root/.cache/torch/hub/checkpoints/convnext_large-ea097f82.pth
100%| | 755M/755M [00:03<00:00, 220MB/s]

```

[15]: checkpoint = torch.load('/kaggle/input/convnext_large/pytorch/default/1/
↳convnext_large_epoch_7.pth')
model.load_state_dict(checkpoint['model_state_dict'])

```

<ipython-input-15-7989bf0c7221>:1: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```

    checkpoint = torch.load('/kaggle/input/convnext_large/pytorch/default/1/convne
xt_large_epoch_7.pth')

```

[15]: <All keys matched successfully>

```

[16]: #Generate predictions and logits for the test set

```

```

model.eval()
test_logits = [] # To store logits
test_pred_classes = []

with torch.no_grad():
    for data, _ in tqdm(test_loader, desc="Generating Test Predictions"):
        data = data.to(device)
        output = model(data) # Raw logits (before softmax)

        # Save logits

```

```

test_logits.extend(output.cpu().numpy()) # Store raw logits

# Get predicted class (0 or 1)
preds = output.argmax(dim=1)
test_pred_classes.extend(preds.cpu().numpy())

# Convert logits to a DataFrame
logits_df = pd.DataFrame(test_logits, columns=['logit_class_0', 'logit_class_1'])
logits_df['id'] = test['id'].values # Add image IDs for reference

# Save logits to a CSV file
logits_df.to_csv('test_logits.csv', index=False)

# Add predictions to the test DataFrame
test['label'] = test_pred_classes
test[['id', 'label']].to_csv('submission.csv', index=False)

print("Test logits saved to 'test_logits.csv'")
print("Test predictions saved to 'submission.csv'")

```

Generating Test Predictions: 100% | 87/87 [04:51<00:00, 3.35s/it]

Test logits saved to 'test_logits.csv'

Test predictions saved to 'submission.csv'

1 highest accuracy of 73.335% using 7th epoch model